AMD Spartan™ UltraScale+ ™ アーキテクチャ: DSP リソース

2025.2

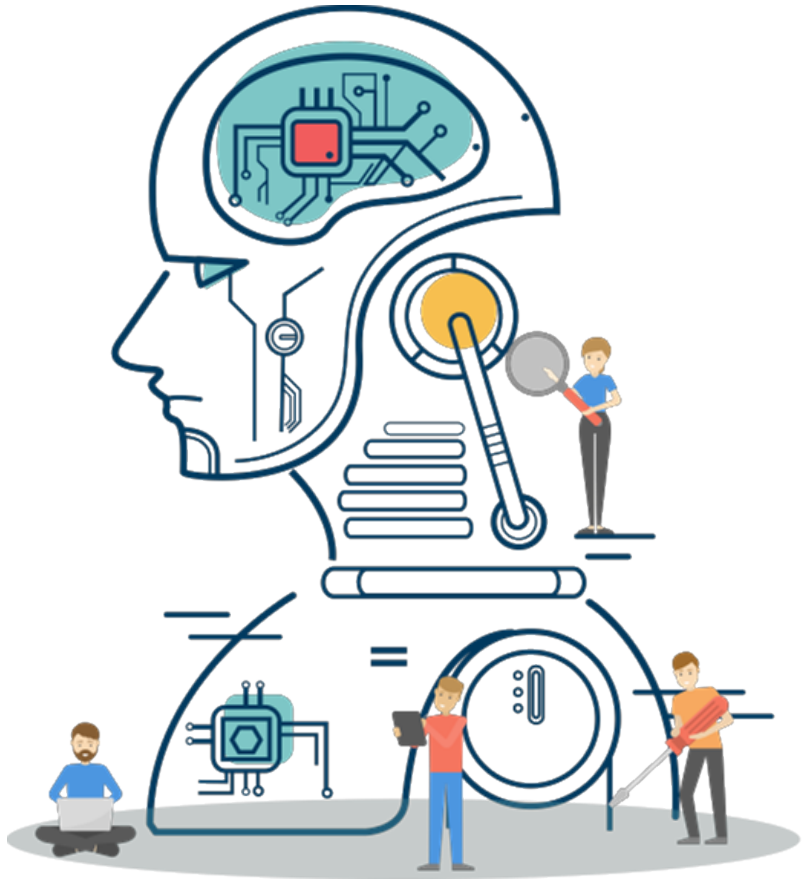
AMD
together we advance_

目標

このモジュールを修了すると、次のことができるようになります。

目的 01

AMD Spartan™ UltraScale+ ™ アーキテクチャの DSP48E2 スライスのアーキテクチャと機能を説明する



© 2026 Advanced Micro Devices, Inc.

AMD
together we advance_

Spartan UltraScale+ デバイスの DSP 機能

1

プログラマブル ロジック デバイス (FPGA):

- DSP アプリケーションに高い効率性を発揮
- カスタムの並列アルゴリズム

2

FPGA 内の DSP リソースにより、乗算器とアキュムレータの性能を最適化

3

Spartan™ UltraScale+ ™ デバイスは、低消費電力の DSP スライスを搭載し、次の特長を備えています。

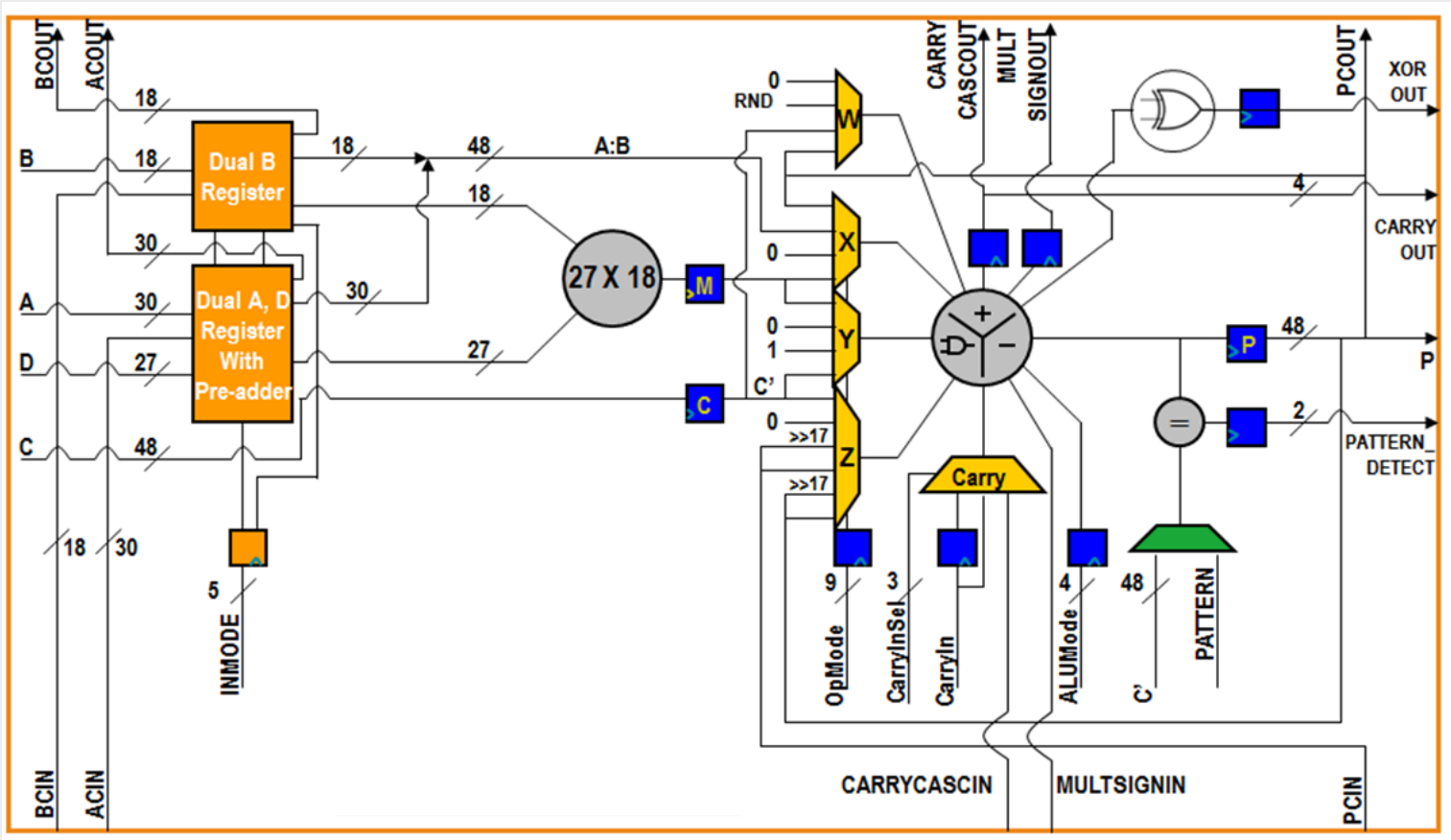
- 高速
- Sモールサイズ
- 設計の柔軟性

4

Spartan UltraScale+ デバイスの DSP スライスは、**DSP** または **DSP48E2** と呼ばれる**DSP48E2プリミティブによって定義されます**

© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

Spartan UltraScale+ アーキテクチャ DSP48E2 スライス



DSP タイルには次のものが含まれます。

- カスケード相互接続を備えた 2 つの DSP スライスにより、タイミングと電力を効率化
- 5 個の CLB または 1 つの BRAM タイルに相当する高さ

DSP48E2スライスは、次のもので構成されます。

- 27 ビットプリ加算器、27x18 2 の補数乗算器、48 ビット ALU
- 4つの48ビットマルチプレクサ、4つの入力加算器/減算器、または2つの入力ロジックユニット

© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

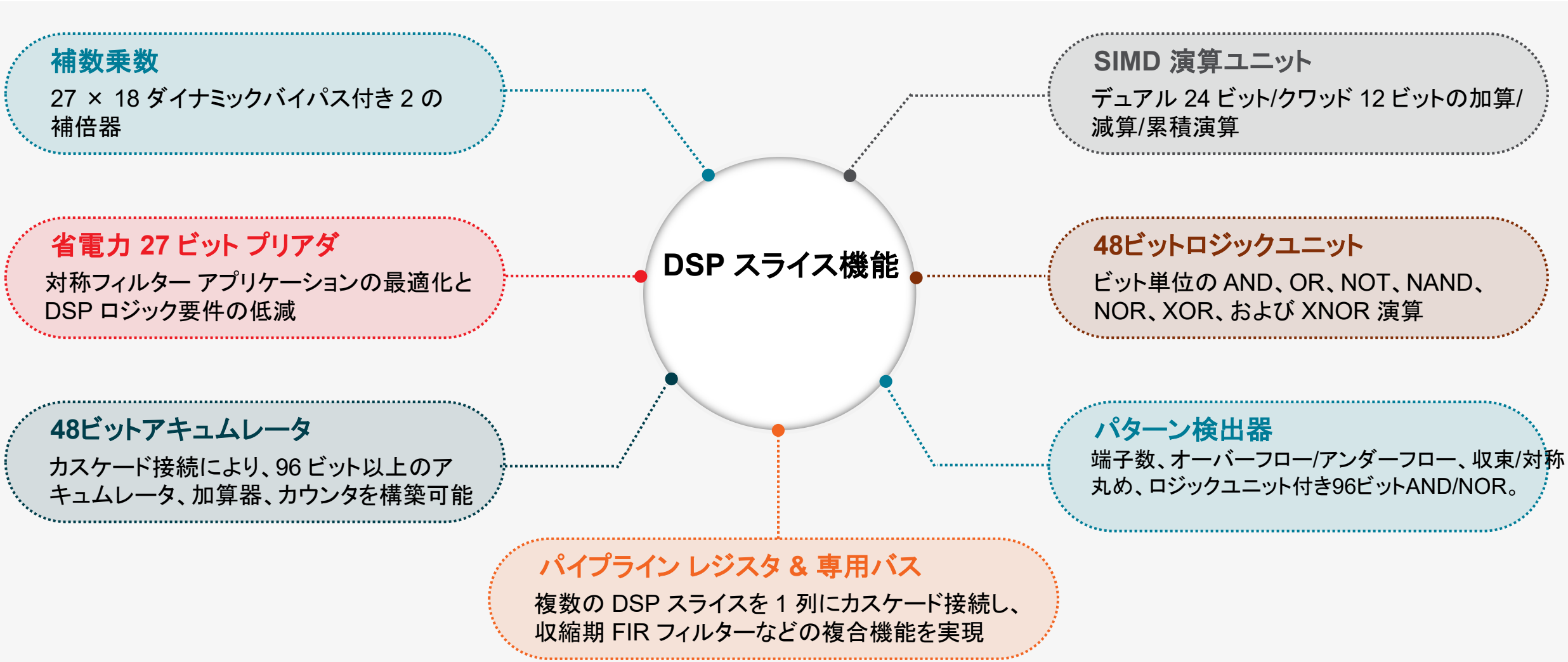
旧アーキテクチャとの比較

DSP48E2スライスは、7 シリーズ FPGA DSP48E1 スライスと下位互換性があり、大幅な進化が加えられています。

- **より広範な機能:** 27x18 ビット乗算器と 27 ビットプリ加算器
- **プリアダの柔軟性の向上:** A/B入力選択とプリアダ出力の二乗
- **WMUX による強化された ALU:** 複雑な MACC または FIR フィルターに対応する最大 4 つのオペランドとアキュムレータ フィードバック
- **幅広いXOR機能:** より大きなXOR関数(最大192ビットカスケード、96ビットシーケンシャル)
- **ユニークな機能:**リセット入力のプログラム可能な反転と自動リセットのクロックイネーブル優先

© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

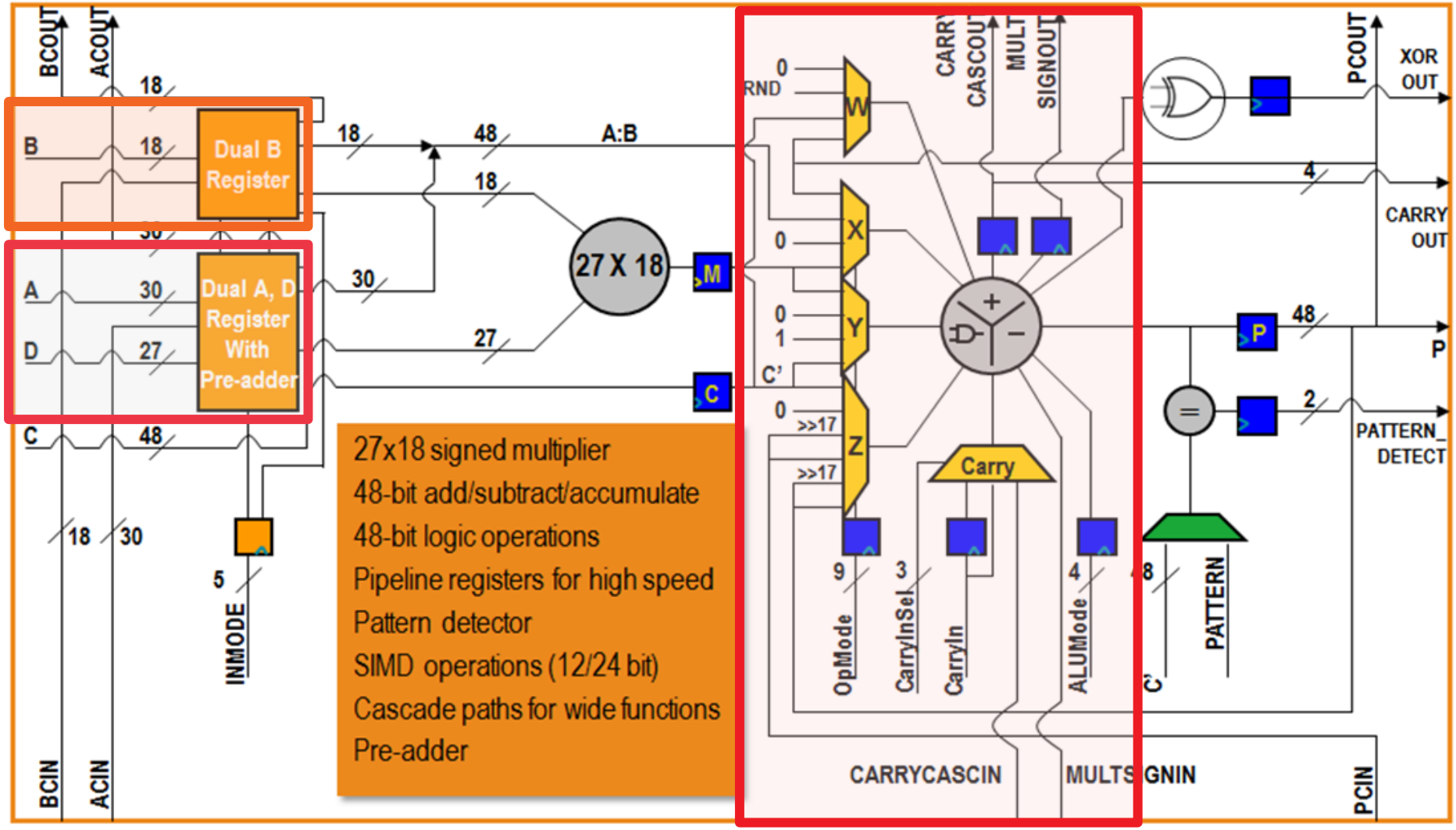
DSP スライス機能のハイライト



© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

DSP48E2スライスコンポーネント

- デュアルBレジスタ
- デュアルA、Dレジスタ
- X、Y、Z、W マルチプレクサ

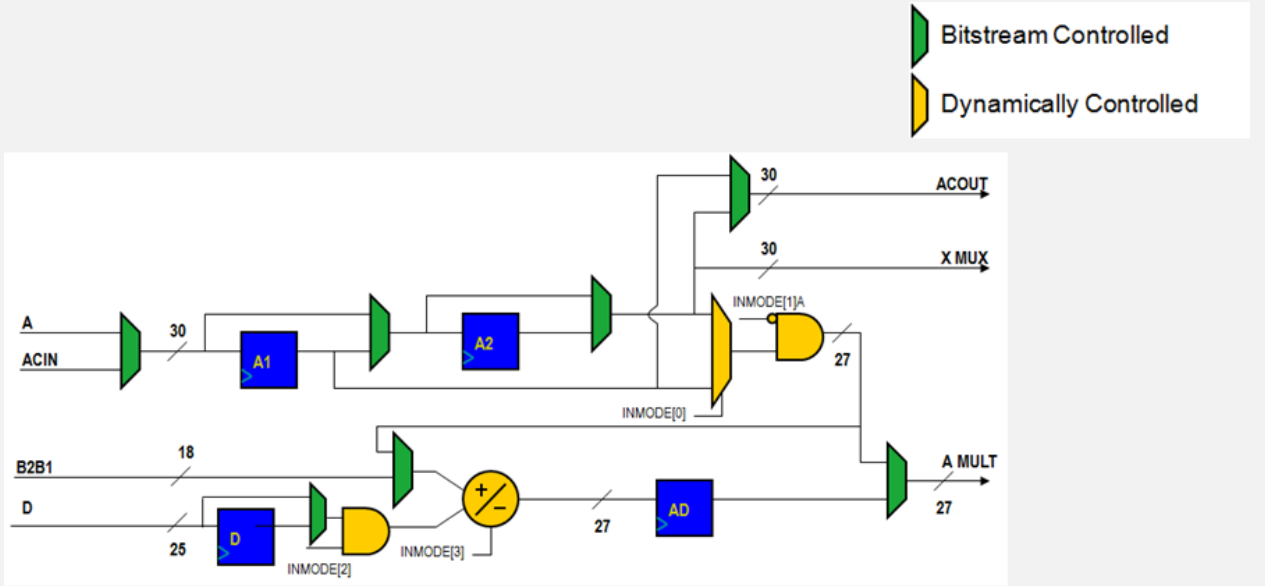


© 2026 Advanced Micro Devices, Inc.

AMD
together we advance_

デュアルA、Dレジスタ

- 乗算器への入力は INMODE[3:0] によって制御されます
  - A1/A2 パイプライン レベルを動的に選択
  - 加算/減算を動的に選択
  - AまたはDのゼロを動的に選択
- MULT出力はBレジスタからB2B1を選択可能
  - B または (D +/- B) の二乗を許可します。
- ACOUTおよびXMUX入力は静的に制御されます



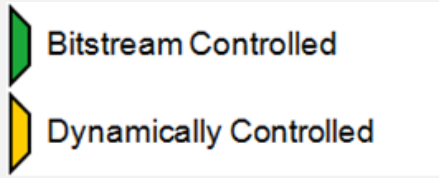
© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

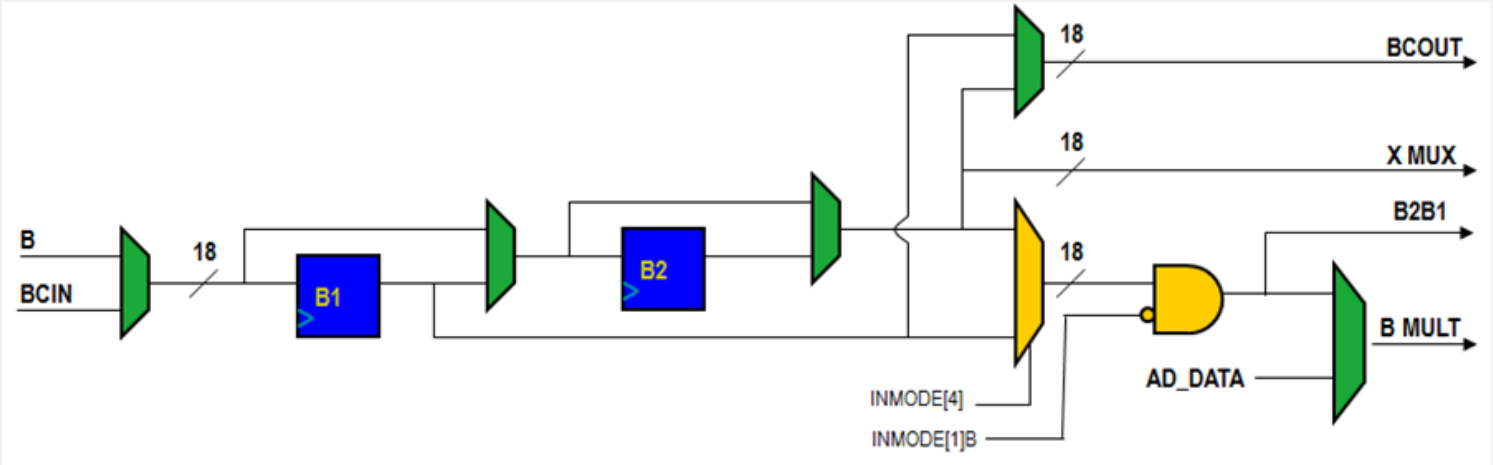
デュアルBレジスタ

乗算器へのB入力はINMODE[4]で制御されます
- B1/B2 パイプライン遅延を動的に選択

B MULTは、ビットストリームオプションを介してAD_DATAから駆動可能
- AD_DATAはデュアルA、Dレジスタから取得されます
  - AD_DATAの二乗を可能にします

X MUXおよびBCOUTカスケード出力は、ビットストリームオプションによって静的に制御されます



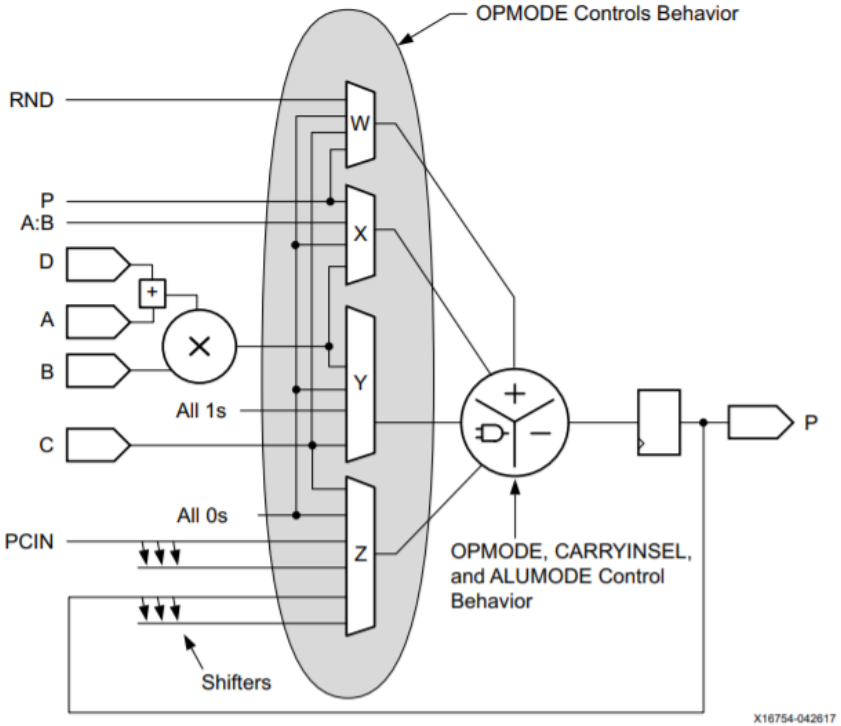


© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

X、Y、Z、W マルチプレクサ

- 加算器/減算器は、X、Y、Z、W、CINオペランドで動作します。
- X、Y、Z、Wマルチプレクサはダイナミックモードをサポート
- 乗算器出力には、Xマルチプレクサと Yマルチプレクサの両方が必要

ALUMODE	Operation
0000	Z + W + X + Y + CIN
0001	-Z + (W + X + Y + CIN) – 1
0010	-Z – W – X – Y – CIN – 1
0011	Z – (W + X + Y + CIN)
Others	Logic Operations



© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

DSP48E2機能

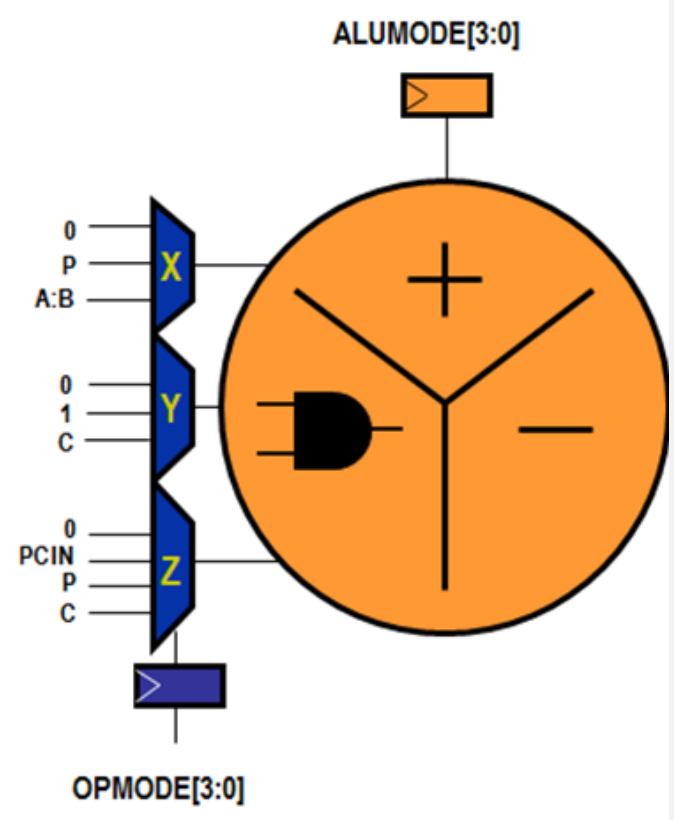
- 2入力ロジック機能
- パターン検出
- ワイドXOR
- SIMD
- カスケード パス

© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

2入力ロジック機能

48 ビット ロジック演算:

- XOR
- XNOR
- AND
- NAND
- OR
- NOR
- NOT



ALUMODEs

Logic Unit Mode	OPMODE[3:2]	ALUMODE[3:0]
X XOR Z	00	0100
X XNOR Z	00	0101
X XNOR Z	00	0110
X XOR Z	00	0111
X AND Z	00	1100
X AND (NOT Z)	00	1101
X NAND Z	00	1110
(NOT X) OR Z	00	1111
X XNOR Z	10	0100
X XOR Z	10	0101
X XOR Z	10	0110
X XNOR Z	10	0111
X OR Z	10	1100
X OR (NOT Z)	10	1101
X NOR Z	10	1110

© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

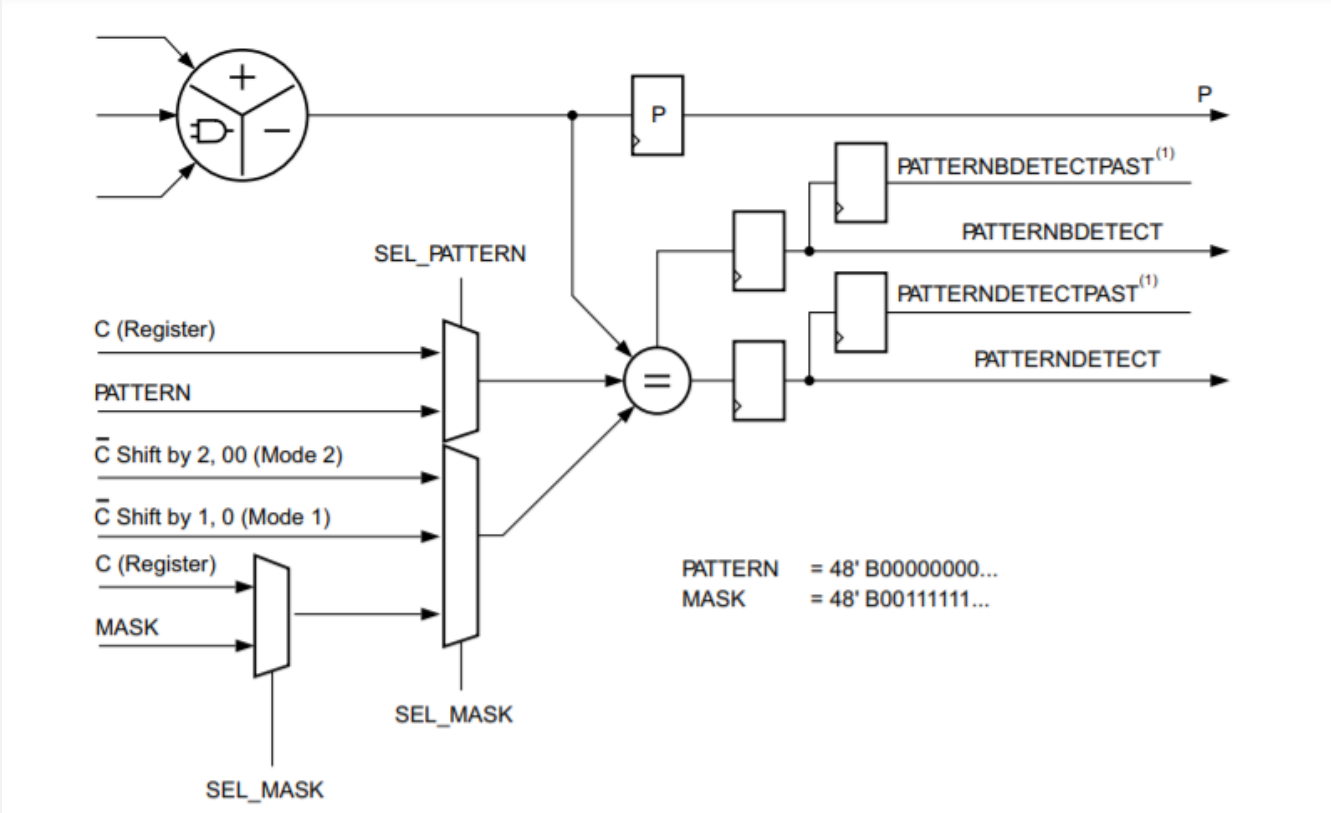
パターン検出

出力はパターン フィルターとマスク フィルターを通過

- 定数または C レジスタ パターン
- PATTERNB は補体を検出します

可能:

- 多精度演算のための対称丸め
- 収束丸め
- 彩度
- アキュムレータ端子数



© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

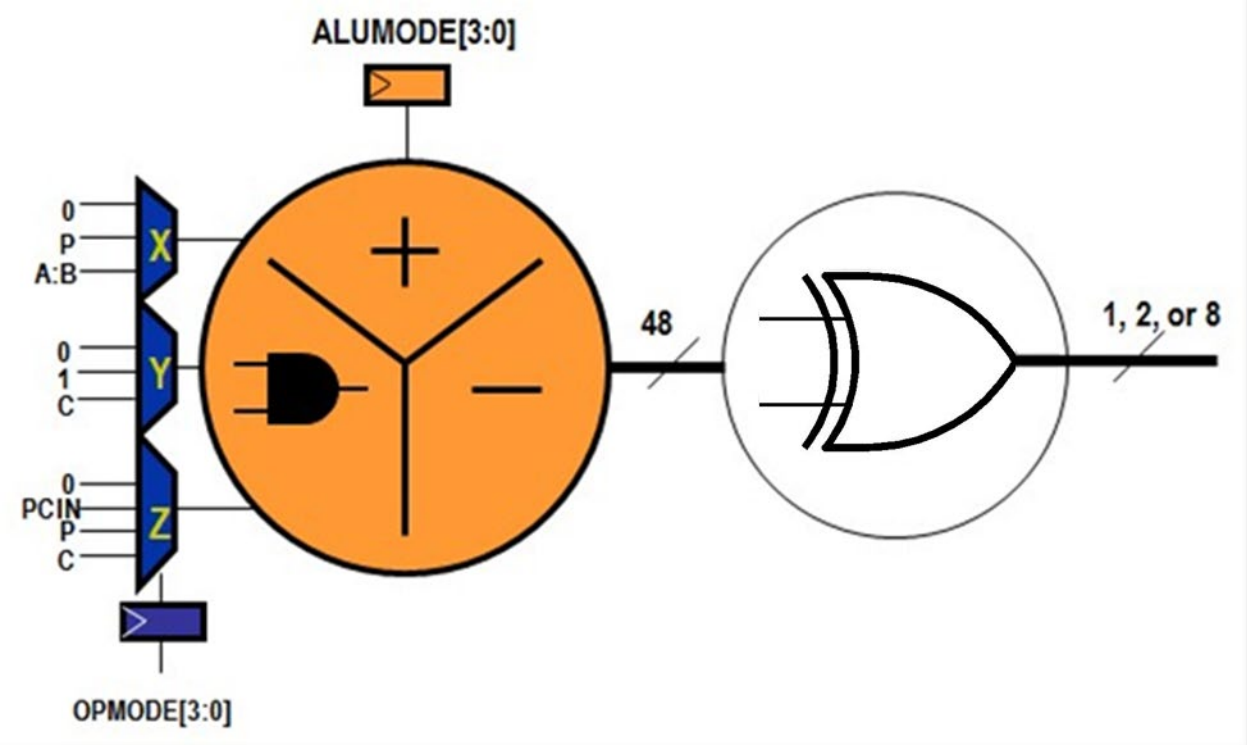
ワイド XOR

96ビット2入力または3入力XOR

- (X XOR Z)
- (X XORおよびXOR Z)

広いXORは出力を低下させるため、次の4つの構成が考えられます。

- 1 x 96 ビット XOR
- 2 x 48 ビット XOR
- 4 x 24 ビット XOR
- 8 x 12 ビット XOR



© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

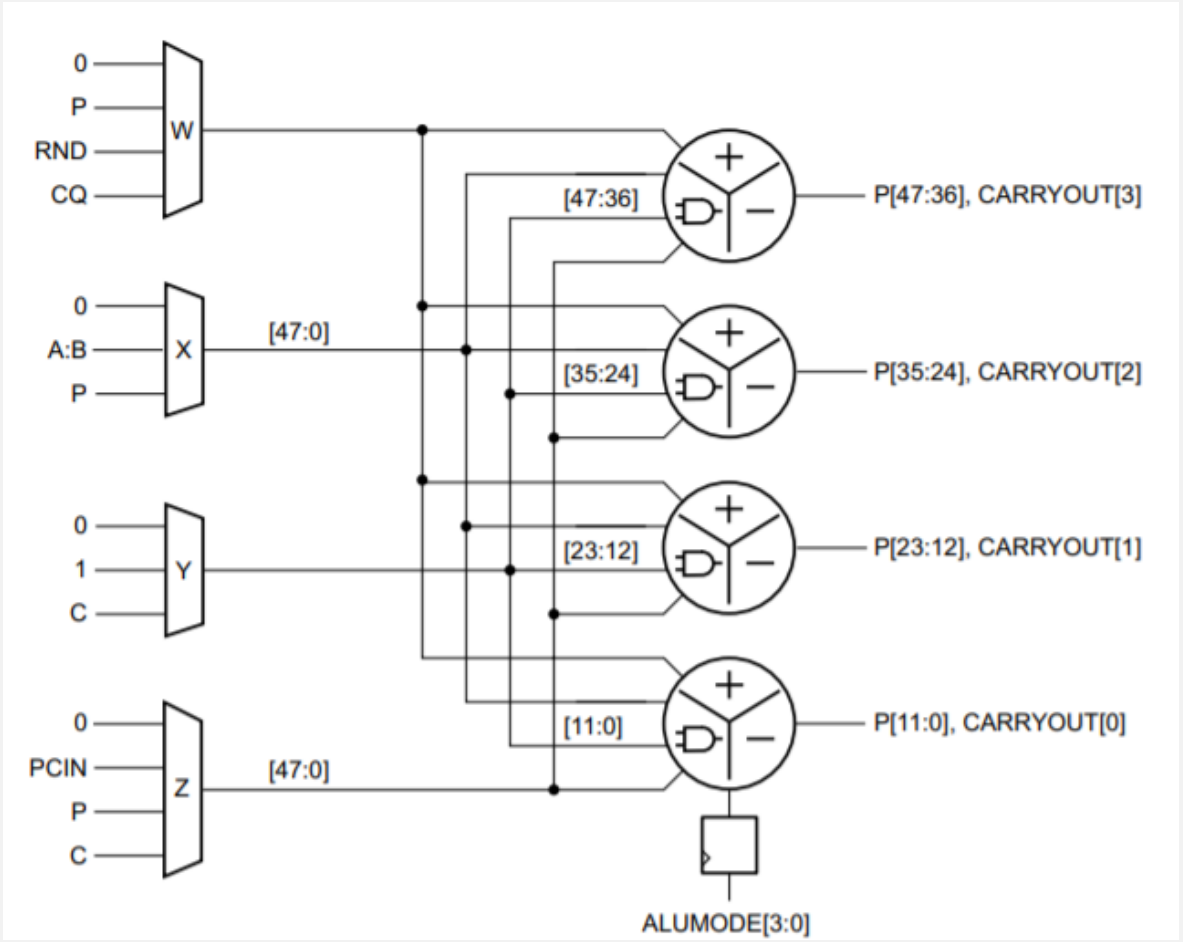
SIMD

2x24 ビットまたは 4x12 ビットに分割された 48 ビット加算器

- 2 つまたは 4 つの独立した追加機能

キャリービットは独立して取り出され、セクション間で無効になります

- キャリービットカスケード

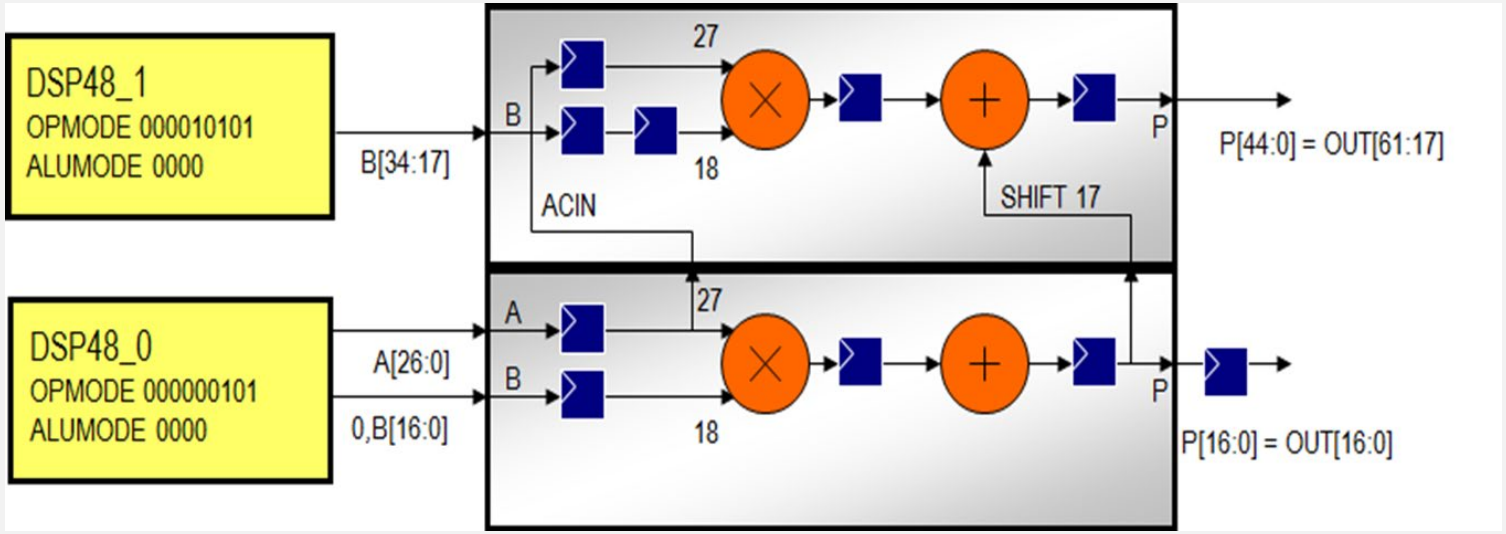


© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

カスケード パス

各 DSP スライスから「北」スライスまで
- A入力、B入力、P出力を行い、
  - P カスケード パスは、上記のスライスごとに 17 ビットシフト可能

一般的な機能を実装する
- より広いアキュムレータ、より広い乗算器、複素乗算器、および FIR フィルター
  - 例: 2 つの DSP48E1 を使用した 35 ビット x 27 ビット乗算器



© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

DSP スライスの推論

16 x 16 乗算器

2入力加算器

ロード可能な MACC

```
-------------------------------------------------------------------
-- Example: 16x16 Multiplier, inputs registered once,
-- outputs twice
--          Matches 1 DSP48 slice
--          OpMode(W,Z,Y,X):Subtract
--            (00,000,01,01):0
-------------------------------------------------------------------
p1 <= a1*b1;

 process (clk) is
 begin
  if clk'event and clk = '1' then
   if rst = '1' then
     a1 <= (others => '0');
     b1 <= (others => '0');
     p  <= (others => '0');
   elsif ce = '1' then
     a1 <= a;
     b1 <= b;
     p  <= p1;
   end if;
  end if;
 end process;
```

```
//////////////////////////////////////////////////////////////////
// Example: 16x16 Multiplier, inputs registered
// once, outputs twice
//          Matches 1 DSP48 slice
//          OpMode(W,Z,Y,X):Subtract
//            (00,000,01,01):0
//////////////////////////////////////////////////////////////////

 assign p1 = a1*b1;

 always @(posedge clk)
  if (rst == 1'b1)
     begin
       a1 <= 0;
       b1 <= 0;
       p <= 0;
     end
  else if (ce == 1'b1)
     begin
       a1 <= a;
       b1 <= b;
       p  <= p1;
     end
```

注 : データ型は符号付きにする必要があります

© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

DSP スライスの推論

16 x 16 乗算器

2入力加算器

ロード可能な MACC

```
-------------------------------------------------------------------
-- Example: 16 bit adder 2 inputs, input and output
-- registered once
--  Mapping to DSP48 should be driven  by timing as
-- DSP48 are limited resources.  The -use_dsp48 XST
-- switch must be set to YES
--  Matches 1 DSP48 slice
--        OpMode(W,Z,Y,X):Subtract
--          (00,000,11,11):0  or
--          (00,011,00,11):0
-------------------------------------------------------------------

p1 <= a1 + b1;

 process (clk) is
 begin
  if clk'event  and clk = '1' then
   if rst = '1' then
     p  <= (others => '0');
     a1 <= (others => '0');
     b1 <= (others => '0');
   elsif ce = '1' then
     a1 <= a;
     b1 <= b;
     p <= p1;
   end if;
  end if;
end process;
```

```
//////////////////////////////////////////////////////////////////
// Example: 16 bit adder 2 inputs, input and output
// registered once
//  Mapping to DSP48 should be driven  by timing as
//  DSP48 are limited resources.  The -use_dsp48 XST
//  switch must be set to YES
//  Matches 1 DSP48 slice
//        OpMode(W,Z,Y,X):Subtract
//          (00,000,11,11):0  or
//          (00,011,00,11):0
//////////////////////////////////////////////////////////////////

assign p1 = a1 + b1;

 always @(posedge  clk)
  if (rst == 1'b1)
     begin
       p  <= 0;
       a1 <= 0;
       b1 <= 0;
     end
  else if (ce == 1'b1)
     begin
       a1 <= a;
       b1 <= b;
       p <= p1;
  end
```


© 2026 Advanced Micro Devices, Inc.

AMD
together we advance_

DSP スライスの推論

16 x 16 乗算器

2入力加算器

ロード可能な MACC

```
-------------------------------------------------------------------------------
-- Example: Loadable Multiply Accumulate with one level
--  of registers
--         Map into 1 DSP48 slice
--         Funtion: OpMode(W,Z,Y,X):Subtract
--         - load     (00,011,00,00):0
--         - mult_acc (00.010,01,01):0
-- Expected mapping:
--         AREG: no, BREG: no, CREG: no, MREG: no, PREG: yes
-------------------------------------------------------------------------------

 with load select p_tmp <= signed(c) when '1',
  p_reg + a1*b1 when others;

 process(clk)
 begin
  if clk'event and clk = '1' then
    if p_rst = '1' then
      p_reg <= (others => '0');
      a1 <= (others => '0');
      b1 <= (others => '0');
    elsif p_ce = '1' then
      p_reg <= p_tmp;
      a1 <= signed(a);
      b1 <= signed(b);
    end if;
  end if;
 end process;

 p <= std_logic_vector(p_reg);
```

```
//////////////////////////////////////////////////////////////////////////////
// Example: Loadable  Multiply Accumulate with one level
//  of registers
//         Map into 1 DSP48 slice
//         Funtion: OpMode(W,Z,Y,X):Subtract
//         - load     (00,011,00,00):0
//         - mult_acc (00.010,01,01):0
// Expected mapping:
//    AREG: no, BREG: no, CREG: no, MREG: no, PREG: yes
//////////////////////////////////////////////////////////////////////////////

 assign p_tmp = load ? c:p + a1*b1;

 always @(posedge  clk)
  if (p_rst  == 1'b1)  begin
    p <= 0;
    a1 <=0;
    b1 <=0;
  end
  else if (p_ce  == 1'b1)  begin
    p <= p_tmp;
    a1 <=a;
    b1 <= b;
  end
```


© 2026 Advanced Micro Devices, Inc.

AMD
together we advance_

IP のサポートと推論

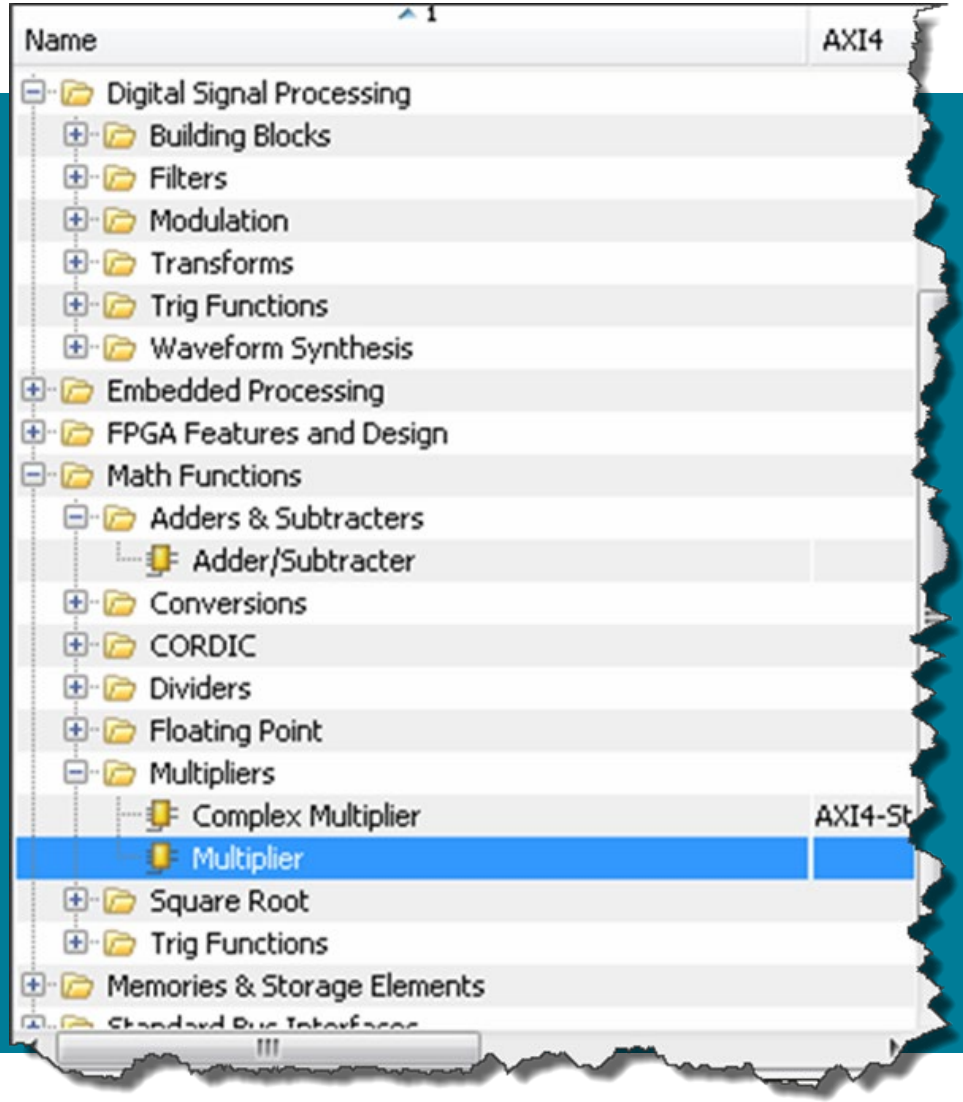
基本関数の推論

- 乗算、乗算累積、その他

IP カタログには数学および DSP 関数が含まれています

DSP プリミティブを直接インスタンス化

設定と接続を担当するユーザー



© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

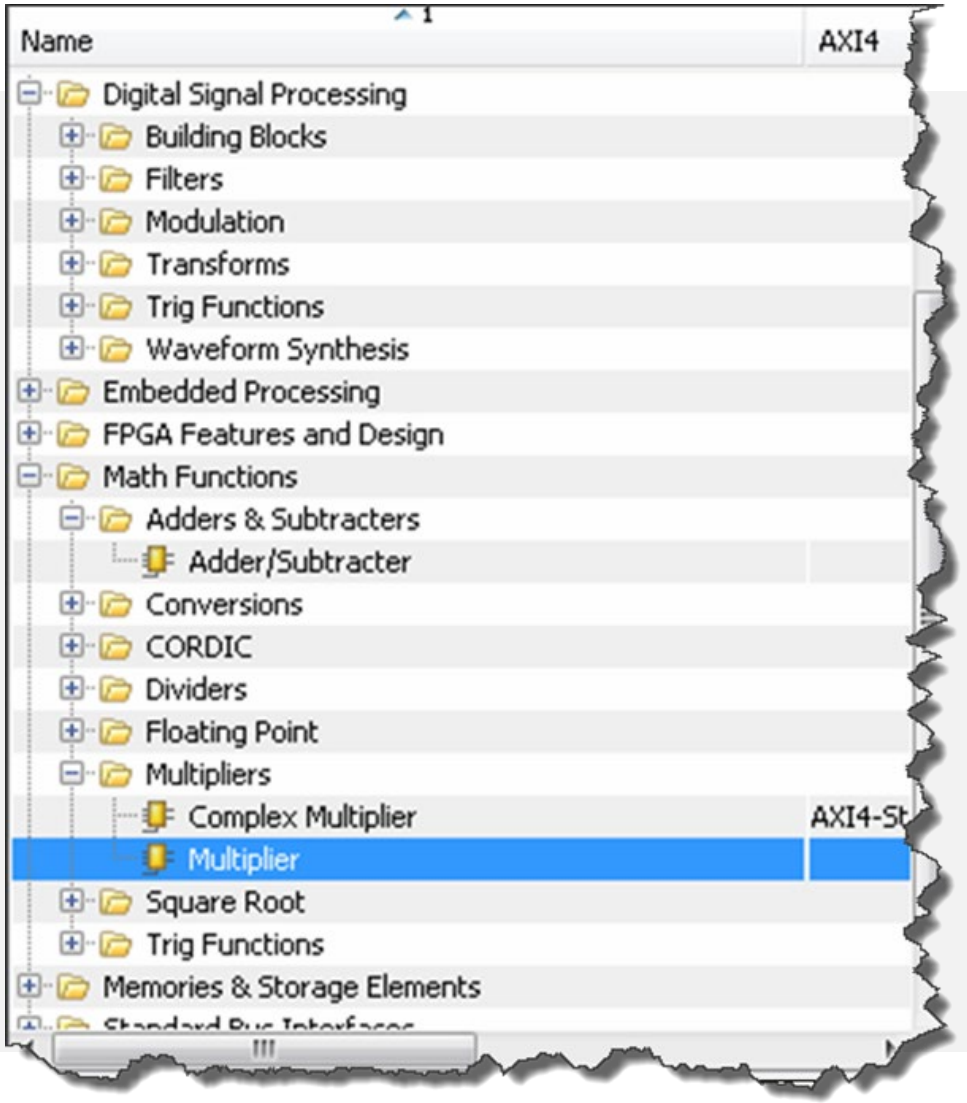
IP カタログの数学演算と DSP 演算

IP カタログで数学/DSP 関数を提供

- ターゲット DSP または LUT
  - 複数の DSP を対象としたビルド操作
- パフォーマンスのトレードオフのための制御パイプライン

言語テンプレートは、インスタンス化テンプレートを提供します

- 100 を超えるポートと属性



© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

知識を応用する

Q Spartan™ UltraScale+ ™ アーキテクチャの DSP スライスの特長は何ですか?
(当てはまるものをすべて選択してください)

- ☐ 27 x 18 2 の補数乗数
- ☐ 48ビット加算/減算/アキュムレータおよび48ビットロジックユニット
- ☐ 異なる電圧レベルで動作する個々のブロック
- ☐ カスケードパスを使用したより広いXORおよびより広い関数
- ☐ SIMD 演算 (12/24 ビット)

© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

知識を応用する

Q DSP スライスにおける SIMD (Single Instruction, Multiple Data) モードはどのように実装されていますか?

- ○ OPMODE入力を動的に変更することにより
- ○ ロジックユニットを独立した動作でより小さなセグメントに分割することにより
- ○ 並列演算に広い ALU を使用することで
- ○ 複数の DSP スライスをカスケード接続することで

© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

まとめ

01 デジタル信号処理 (DSP) 機能により、超高速の数学演算が可能
- 高速乗算演算と乗算累積演算
- 高性能デジタル信号処理 (DSP) アプリケーション向けの優れたリソース

02 DSP リソースを推論またはインスタンス化可能

03 DSP機能はIPカタログからも入手できます

© 2026 Advanced Micro Devices, Inc.
AMD
together we advance_

免責条項および権利帰属

ここに記載されている情報は、情報提供のみを目的としており、事前通知なしで変更される場合があります。この資料の作成時には確認を重ねているものの、技術的な誤りや欠落、誤記などが含まれる可能性があり、AMD は当該情報の更新または修正の義務を負いません。Advanced Micro Devices, Inc. は、この資料の内容の正確性または完全性に関していかなる表明または保証も行わず、ここに記載される AMD ハードウェア、ソフトウェア、その他の製品の操作または使用に関して、非侵害、商品性、特定の目的への適合性の黙示的な保証を含め、いかなる種類の責任も一切負わないものとします。この資料は、黙示的あるいは禁反言で生じるものを含め、いかなる知的財産権へのライセンス付与を行うものではありません。AMD 製品の購入または使用に適用される条件および制限は、当事者間で締結された契約または AMD 標準売買条件に規定されているとおりです。GD-18。

©2026 Advanced Micro Devices, Inc.All rights reserved.AMD、AMD Arrow ロゴ、Spartan、UltraScale+、Vivado、およびその組み合わせは、Advanced Micro Devices, Inc. の商標です。本ドキュメントに使用されるその他の商品名は情報提供のみを目的としており、各所有者の商標である可能性があります。一部の AMD テクノロジでは、サードパーティによる有効化またはアクティブ化が必要になる場合があります。サポートされる機能はオペレーティング システムによって異なる場合があります。具体的な機能については、システム メーカーにお問い合わせください。完全に安全なテクノロジや製品はありません。

© 2026 Advanced Micro Devices, Inc.


AMD